ROMs

Where to put your games, which file formats each core accepts, and how to turn a folder of files into playlists with box art.

Contents

1. The ROM folder
    - › Keeping ROMs on an external drive
    - › Example layout
2. Folders, cores and file extensions
3. Zipped or not?
4. CD and DVD games
    - › CHD (recommended)
    - › CUE and BIN
    - › M3U for multi-disc games
5. Building playlists
    - › With the playlist tool (recommended)
    - › With RetroArch's scanner
6. Box art and thumbnails

The ROM folder

`install.sh` creates one folder per system inside `~/Games/roms`, or inside the folder you passed with `--roms`. The folder names come from the first column of `cores.tsv`. Put each game you own in the folder of its system.

WARNING

Do not keep ROMs in `~/Documents` or `~/Desktop` if iCloud Drive syncs them. macOS can move files to the cloud to free space, and RetroArch then stalls while it waits for them to download. `~/Games/roms` is outside iCloud.

Keeping ROMs on an external drive

Large collections, such as a MAME set or disc games, can live on an external drive while the smaller systems stay on the Mac. `~/Games/roms` remains the ROM folder; each system kept on the drive is a link from `~/Games/roms/<system>` to its folder on the drive. Games on the drive then appear in RetroArch and ES-DE while it is connected, and simply disappear while it is not, without *file is missing* errors.

Choose exFAT if the drive will also be used on Windows or Linux; all three read and write it. Use **APFS** if the drive is only for Macs. Avoid NTFS, which macOS can only read. This command shows the format of a drive called *BACKUP*:

```
diskutil info /Volumes/BACKUP | grep "File System Personality"
```

Put the games on the drive in one folder per system, named as in `cores.tsv`, for example `/Volumes/BACKUP/roms/arcade`. Then, with the drive connected, run the installer. It replaces each empty local system folder that the drive also has with a link to it (a folder with games of its own is left alone), builds the playlists, including the systems on the drive, and copies them into ES-DE. It also copies BIOS files missing from the system folder out of the newest `backup.sh` backup on the drive.

```
./install.sh
```

How the two programs follow the drive:

- **RetroArch:** the playlist tools write the complete playlists to `~/Documents/RetroArch/playlists-full`. `tools/sync_playlists.py` copies them into RetroArch's playlist folder with only the games whose files exist, and drops a playlist that has none. The *RetroArch CRT* launcher runs it every time it opens RetroArch, so open RetroArch from the launcher (or from ES-DE). Changes you make in RetroArch to such a playlist, like renaming an entry, are replaced the next time it runs.
- **ES-DE:** it reads the folders each time it starts. A system whose link points at a disconnected drive is not shown, and the systems stored on the Mac always are. Keep at least one system on the Mac: with no games at all, ES-DE opens with its *no games found* dialog.

IMPORTANT

The links point at the drive's volume name, `/Volumes/BACKUP`. Keep that name. Connect the drive before opening ES-DE or RetroArch; if you connect it later, quit and reopen them. Saves and states stay on the Mac, so they work with or without the drive.

Copy, clean up, then compare. This copies a folder of ROMs to the drive, skipping Finder's `.DS_Store` files.

```
rsync -rt --exclude '.DS_Store' ~/Games/roms/arcade/
/Volumes/BACKUP/roms/arcade/
```

exFAT cannot store macOS file metadata, so macOS writes a hidden companion file next to every file copied to the drive, for example `._sf2.zip` next to `sf2.zip`. They hold only metadata such as the quarantine flag, never game data. `./install.sh` deletes them in the ROM folders, including folders linked to a drive, and ES-DE is set to ignore files starting with a dot. To delete them by hand elsewhere on the drive, run this twice, because on exFAT a single pass can skip some entries while it deletes.

```
find /Volumes/BACKUP/roms -name '._*' -type f -delete
find /Volumes/BACKUP/roms -name '._*' -type f -delete
```

This compares the copy with the original, file by file. No output means they are identical, and only then is it safe to delete the original.

```
diff -rq ~/Games/roms/arcade /Volumes/BACKUP/roms/arcade
```

WARNING

exFAT has no journal. Always eject the drive in the Finder before unplugging it, or files that were being written can be corrupted.

Example layout

```
~/Games/roms/
├── arcade/                    MAME zips, never unzipped (see the
MAME guide)
│   ├── pacman.zip
│   ├── sf2.zip
│   └── neogeo.zip             BIOS zip, next to the games that
need it
├── snes/
│   ├── Super Metroid (Japan, USA).zip
│   └── Chrono Trigger (USA).sfc
├── gba/
│   └── Metroid Fusion (USA).gba
├── genesis/
│   └── Sonic the Hedgehog (USA, Europe).md
├── psx/
│   ├── Castlevania - Symphony of the Night (USA).chd
│   ├── Final Fantasy VII (USA).m3u
│   ├── Final Fantasy VII (USA) (Disc 1).chd
│   ├── Final Fantasy VII (USA) (Disc 2).chd
│   └── Final Fantasy VII (USA) (Disc 3).chd
├── psp/
│   └── Patapon (USA).iso
└── gamecube/
    └── Metroid Prime (USA).rvz
```

This lists every folder the installer created, so you can check the names.

```
ls ~/Games/roms
```

Folders, cores and file extensions

Each folder is played with the default core listed below. The extensions are the ones the core declares in its info file, so RetroArch only offers files with these extensions. Alternate cores, such as bsnes for SNES or Beetle PSX HW for PlayStation, are installed too; you can pick one per game with **Set Core Association** in the game's playlist entry.

Folder	System	Core	Extensions the core accepts
`atari2600`	Atari 2600	Stella	`.a26`, `.bin`
`atari5200`	Atari 5200	Atari800	`.xfd`, `.atr`, `.dcm`, `.cas`, `.bin`, `.a52`, `.zip`, `.atx`, `.car`, `.rom`, `.com`, `.xex`, `.m3u`
`atari7800`	Atari 7800	ProSystem	`.a78`, `.bin`, `.cdf`
`lynx`	Atari Lynx	Handy	`.lnx`, `.lyx`, `.o`
`jaguar`	Atari Jaguar	Virtual Jaguar	`.j64`, `.jag`, `.rom`, `.abs`, `.cof`, `.bin`, `.prg`
`coleco`	ColecoVision	Gearcoleco	`.col`, `.cv`, `.bin`, `.rom`
`intellivision`	Intellivision	FreeIntv	`.int`, `.bin`, `.rom`
`odyssey2`	Odyssey² / Videopac	O2EM	`.bin`
`vectrex`	Vectrex	vecx	`.bin`, `.vec`
`nes`	NES / Famicom	Nestopia	`.nes`, `.fds`, `.unf`, `.unif`
`fds`	Famicom Disk System	Nestopia	`.nes`, `.fds`, `.unf`, `.unif`
`snes`	Super Nintendo	Snes9x	`.smc`, `.sfc`, `.swc`, `.fig`, `.bs`, `.st`
`n64`	Nintendo 64	Mupen64Plus-Next	`.n64`, `.v64`, `.z64`, `.ndd`, `.bin`, `.u1`
`gb`	Game Boy	Gambatte	`.gb`, `.gbc`, `.dmg`
`gbc`	Game Boy Color	Gambatte	`.gb`, `.gbc`, `.dmg`
`gba`	Game Boy Advance	mGBA	`.gb`, `.gbc`, `.gba`
`nds`	Nintendo DS	melonDS DS	`.nds`, `.ids`, `.dsi`
`vb`	Virtual Boy	Beetle VB	`.vb`, `.vboy`, `.bin`
`pokemini`	Pokémon mini	PokeMini	`.min`
`gamecube`	GameCube	Dolphin	`.gcm`, `.iso`, `.wbfs`, `.ciso`, `.gcz`, `.elf`, `.dol`, `.dff`, `.tgc`, `.wad`, `.rvz`, `.m3u`, `.wia`
`sg1000`	SG-1000	Genesis Plus GX	`.mdx`, `.md`, `.smd`, `.gen`, `.bin`, `.cue`, `.iso`, `.sms`, `.bms`, `.gg`, `.sg`, `.68k`, `.sgd`, `.chd`, `.m3u`
`mastersystem`	Master System	Genesis Plus GX	`.mdx`, `.md`, `.smd`, `.gen`, `.bin`, `.cue`, `.iso`, `.sms`, `.bms`, `.gg`, `.sg`, `.68k`, `.sgd`, `.chd`, `.m3u`
`gamegear`	Game Gear	Genesis Plus GX	`.mdx`, `.md`, `.smd`, `.gen`, `.bin`, `.cue`, `.iso`, `.sms`, `.bms`, `.gg`, `.sg`, `.68k`, `.sgd`, `.chd`, `.m3u`
`genesis`	Mega Drive / Genesis	Genesis Plus GX	`.mdx`, `.md`, `.smd`, `.gen`, `.bin`, `.cue`, `.iso`, `.sms`, `.bms`, `.gg`, `.sg`, `.68k`, `.sgd`, `.chd`, `.m3u`
`segacd`	Mega-CD / Sega CD	Genesis Plus GX	`.mdx`, `.md`, `.smd`, `.gen`, `.bin`, `.cue`, `.iso`, `.sms`, `.bms`, `.gg`, `.sg`, `.68k`, `.sgd`, `.chd`, `.m3u`
`32x`	32X	PicoDrive	`.bin`, `.gen`, `.smd`, `.md`, `.32x`, `.cue`, `.iso`, `.chd`, `.sms`, `.gg`, `.sg`, `.sc`, `.m3u`, `.68k`, `.sgd`, `.pco`
`saturn`	Saturn	Beetle Saturn	`.ccd`, `.chd`, `.cue`, `.toc`, `.m3u`
`dreamcast`	Dreamcast	Flycast	`.chd`, `.cdi`, `.elf`, `.bin`, `.cue`, `.gdi`, `.lst`, `.zip`, `.dat`, `.7z`, `.m3u`
`psx`	PlayStation	SwanStation	`.exe`, `.psexe`, `.cue`, `.bin`, `.img`, `.iso`, `.chd`, `.pbp`, `.ecm`, `.mds`, `.psf`, `.m3u`
`psp`	PSP	PPSSPP	`.elf`, `.iso`, `.cso`, `.prx`, `.pbp`, `.chd`
`ps2`	PlayStation 2	LRPS2	`.elf`, `.iso`, `.ciso`, `.cue`, `.bin`, `.gz`, `.chd`, `.cso`, `.zso`, `.mdf`, `.nrg`, `.dump`, `.img`, `.m3u`
`pce`	PC Engine / TurboGrafx-16	Beetle PCE Fast	`.pce`, `.cue`, `.ccd`, `.chd`, `.toc`, `.m3u`
`pcecd`	PC Engine CD	Beetle PCE Fast	`.pce`, `.cue`, `.ccd`, `.chd`, `.toc`, `.m3u`
`pcfx`	PC-FX	Beetle PC-FX	`.cue`, `.ccd`, `.toc`, `.chd`
`wswan`	WonderSwan	Beetle WonderSwan	`.ws`, `.wsc`, `.pc2`, `.pcv2`
`wswanc`	WonderSwan Color	Beetle WonderSwan	`.ws`, `.wsc`, `.pc2`, `.pcv2`
`ngp`	Neo Geo Pocket	Beetle NeoPop	`.ngp`, `.ngc`, `.ngpc`, `.npc`
`ngpc`	Neo Geo Pocket Color	Beetle NeoPop	`.ngp`, `.ngc`, `.ngpc`, `.npc`
`3do`	3DO	Opera	`.iso`, `.bin`, `.chd`, `.cue`
`msx`	MSX	blueMSX	`.rom`, `.ri`, `.mx1`, `.mx2`, `.dsk`, `.col`, `.sg`, `.sc`, `.sf`, `.cas`, `.m3u`
`c64`	Commodore 64	VICE x64sc	`.d64`, `.d71`, `.d80`, `.d81`, `.d82`, `.g64`, `.g41`, `.x64`, `.t64`, `.tap`, `.prg`, `.p00`, `.crt`, `.bin`, `.zip`, `.gz`, `.d6z`, `.d7z`, `.d8z`, `.g6z`, `.g4z`, `.x6z`, `.cmd`, `.m3u`, `.vfl`, `.vsf`, `.nib`, `.nbz`, `.d2m`, `.d4m`
`supervision`	Watara Supervision	Potator	`.bin`, `.sv`
`arcade`	Arcade (MAME)	MAME 2003-Plus	`.zip`

`.zip` is accepted for every cartridge system even where it is not listed, because RetroArch unpacks the archive before handing the game to the core (see the next section).

Zipped or not?

Keep these zipped (or zip them if you like):

- **Cartridge systems:** NES, SNES, Game Boy, GBA, Mega Drive, Master System, PC Engine, Atari, WonderSwan and similar. These files are small. RetroArch unpacks them in memory in a fraction of a second, and `tools/make_playlists.py` reads the CRC inside the zip to identify the game.
- **Arcade (MAME):** always zipped. MAME looks inside each zip for the ROM chips by name and checksum. See the MAME and arcade guide.

Do not zip these:

- **Disc systems** (PlayStation, PS2, Saturn, Sega CD, PC Engine CD, Dreamcast, 3DO, PC-FX): disc images are hundreds of megabytes, so unpacking them on every launch is slow, and `.cue` files would point to tracks that are not on disk. Use `.chd` instead; it is compressed and loads directly.
- **PSP, GameCube and Nintendo DS:** PPSSPP, Dolphin and melonDS DS read their own compressed formats (`.cso` / `.chd`, `.rvz` / `.gcz`, plain `.nds`) directly. A `.zip` only adds a slow unpacking step.

NOTE

Leave `.7z` and `.rar` archives aside. Unpack them and, for cartridge games, re-zip them as `.zip` if you want them compressed.

CD and DVD games

Disc games come as a set of files describing one disc. There are three common shapes.

CHD (recommended)

One `.chd` file per disc, losslessly compressed, usually 30–60% smaller than the original. Every disc core in this setup reads it: SwanStation, Beetle PSX HW, Beetle Saturn, Genesis Plus GX, PicoDrive, Beetle PCE FAST, Beetle PC-FX, Flycast, Opera, PPSSPP and LRPS2.

This installs `chdman`, the converter from the MAME project, with Homebrew.

```
brew install rom-tools
```

This converts one CD image from `.cue` / `.bin` to `.chd`. The original files are not changed, so delete them yourself once the `.chd` works.

```
chdman createcd -i "Game (USA).cue" -o "Game (USA).chd"
```

For PS2 and PSP games, which are DVD or UMD images in `.iso` format, use `createdvd` instead.

```
chdman createdvd -i "Game (USA).iso" -o "Game (USA).chd"
```

CUE and BIN

A `.cue` file is a small text file listing the `.bin` tracks of a disc. Keep the `.cue` and all its `.bin` files together in the same folder, with the names the `.cue` expects, and open the `.cue`. The playlist tool lists only the `.cue`, never its `.bin` tracks.

M3U for multi-disc games

Games with several discs need a `.m3u` playlist so that you can swap discs from **Quick Menu → Disc Control** without losing your progress. It is a text file with one disc file name per line, in order.

This creates the `.m3u` for a three-disc game whose discs are `.chd` files in the same folder.

```
cd ~/Games/roms/psx
printf '%s\n' "Final Fantasy VII (USA) (Disc 1).chd" "Final Fantasy
VII (USA) (Disc 2).chd" "Final Fantasy VII (USA) (Disc 3).chd" >
"Final Fantasy VII (USA).m3u"
```

Open the `.m3u`, not the individual discs. The playlist tool lists only the `.m3u` and hides the discs it references, and the memory card stays the same across discs.

TIP

When the game asks for the next disc, open the Quick Menu (F1), choose **Disc Control → Eject Disc**, pick the next disc with **Current Disc Index**, then **Insert Disc**.

Building playlists

Playlists are the lists of games that appear as icons in the XMB menu, one per system. The tools write them to `~/Documents/RetroArch/playlists-full`, and `tools/sync_playlists.py` copies them to `~/Documents/RetroArch/playlists`, the folder RetroArch reads, keeping only the games whose files are present. Each one is named after the system's libretro database name, for example `Nintendo - Super Nintendo Entertainment System.lpl`. That exact name is what makes box art work.

With the playlist tool (recommended)

`./install.sh` runs it for you at the end, together with the arcade tool and the ES-DE step, so after adding games you can simply run `./install.sh` again. To run only this step, use this from the repository folder. It builds one playlist per system folder that contains games, and downloads the box art, screenshots and title screens.

```
python3 tools/make_playlists.py
```

If your games are not in `~/Games/roms`, tell it where they are.

```
python3 tools/make_playlists.py --roms /Volumes/Games/roms
```

This builds the playlists without downloading any images, which is useful offline. RetroArch still fetches the images later, when you browse the playlist.

```
python3 tools/make_playlists.py --no-thumbnails
```

How the tool identifies each game:

1. **By CRC.** For zips it reads the CRC stored inside the archive. For cartridge-sized files (up to 64 MB) it computes the CRC of the file. The CRC is looked up in the official libretro-database DAT for the system, so a file called `smw.sfc` still appears as *Super Mario World*.
2. **By file name**, when the CRC is unknown or the file is a disc image. Disc images are too large to checksum quickly, so name them as in the DAT, with or without the region tags: `Crash Bandicoot (USA).chd` and `Crash Bandicoot.chd` both work.

Games are listed by a clean title: the official name without region, language and revision tags, with a trailing *The* moved to the front. *Legend of Zelda, The - The Minish Cap (Europe) (En,Fr,De,Es,It)* becomes *The Legend of Zelda - The Minish Cap*. A file you named yourself (capitals or spaces, no brackets) keeps its own name, so `Pokemon Rubi.zip` is listed as *Pokemon Rubi*. Save files are named after the ROM file, so renaming a game also means renaming its `.srm` and `.state` files in `~/Documents/RetroArch/saves` and `states`.

The DATs are downloaded once from the libretro-database repository and cached in `~/Library/Caches/retroarch-macos-arcade-setup`. Arcade games are handled by a separate tool, described in the MAME and arcade guide.

Quit and reopen RetroArch after building playlists, so the menu reloads them.

With RetroArch's scanner

RetroArch can also scan folders itself. This setup enables `scan_serial_and_crc`, so it checks both disc serials and CRCs.

1. Go to **Main Menu → Import Content → Scan Directory**.
2. Browse to `~/Games/roms/snes` (or the system you want) and choose **‹Scan This Directory›**.
3. When the scan finishes, the new playlist appears in the XMB.

The scanner only adds games it recognises in its database. Games with unusual dumps are skipped, while the playlist tool adds them with their file name. Do not scan the `arcade` folder: use `tools/make_arcade_playlist.py` instead, which picks the right MAME core for each game.

Box art and thumbnails

This setup turns on `network_on_demand_thumbnails`. Whenever you highlight a game, RetroArch downloads its images from `thumbnails.libretro.com` if they are missing.

- **Right side:** box art (`menu_thumbnails = 3`).
- **Left side:** an in-game screenshot (`menu_left_thumbnails = 1`).

Images are saved in `~/Library/Application Support/RetroArch/thumbnails/<playlist name>/`, inside `Named_Boxarts`, `Named_Snaps` and `Named_Titles`. They are not part of the repository or of the backup, because they can always be downloaded again.

To download all images for a playlist in one go, open **Main Menu → Online Updater → Playlist Thumbnails Updater** and choose the playlist.

NOTE

Thumbnails are matched by the game's label in the playlist, with `& * / : < > ? \ | "` replaced by `_`. The server stores images under official names, so RetroArch's own downloads (on demand and the **Playlist Thumbnails Updater**) only find them for labels that are official names. `tools/make_playlists.py` downloads them by official name and saves them under each game's clean title, so rebuild the playlist with it when a game shows no image. Some games simply have no artwork on the server.